Page No
Date

Turings Machine

A Turings Machine (TM) is a mathematical model of computation describing an abstract machine, that manipulates symbols on a strip of tape according to a table of rules.

A TM is expressed as a 7 tuple :

Q : FNES of states

q_0 : initial state

F : FNES of final states

Σ : FNES of input symbols (which are written on tape)

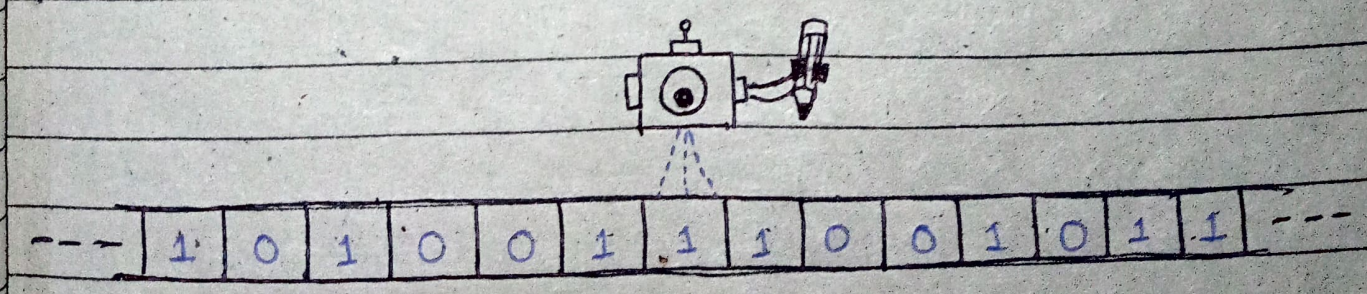
T : FNES of tape symbols (which can be written on tape)

b/B : blank symbol

δ : FNES of transitions which maps

$$(Q \times T) \longrightarrow (Q \times T \times \{L, R\})$$

Transition function for a TM : It takes two inputs — current state and current tape alphabet (pointed by head pointer) and gives three outputs — next state, update tape symbol and movement of head pointer (L or R).



Page No.
Date

Q Construct a turing machine for $L = \{0^n 1^n \mid n >= 1\}$

$Q = \{q_0, q_1, q_2, q_3\}$ // q_0 is initial state.
$F = \{q_4\}$
$\Sigma = \{0, 1\}$
$\Gamma = \{0, 1, X, Y, B\}$ // B is the blank symbol

δ	0	1	X	Y	B
q_0	q_1, X, R	–	–	q_3, Y, R	–
q_1	$q_1, 0, R$	q_2, Y, L	–	q_1, Y, R	–
q_2	$q_2, 0, L$	–	q_0, X, R	q_2, Y, L	–
q_3	–	–	–	q_3, Y, R	q_4, B, R

eg: Let the string be "0011"

q_0 B [0] 0 1 1 B
q_1 B X [0] 1 1 B
q_1 B X 0 [1] 1 B
q_2 B X [0] Y 1 B
q_2 B [X] 0 Y 1 B
q_0 B X [0] Y 1 B
q_1 B X X [Y] 1 B
q_1 B X X Y [1] B
q_2 B X X [Y] Y B
q_2 B X [X] Y Y B

q_0 B X X [Y] Y B

q_3 B X X Y [Y] B

q_3 B X X Y Y [B]

q_4 B X X Y [Y] B

now, $\delta(q_4, Y)$ doesn't exist, so the machine stops at $q_4 \in F$

hence the string "0011" is accepted.

eg: Let the string be "00111"

q_0 B [0] 0 1 1 1 B

q_1 B X [0] 1 1 1 B

q_1 B X 0 [1] 1 1 B

q_2 B X [0] Y 1 1 B

q_2 B [X] 0 Y 1 1 B

q_0 B X [0] Y 1 1 B

q_1 B X X [Y] 1 1 B

q_1 B X X Y [1] 1 B

q_2 B X X [Y] Y 1 B

q_2 B X [X] Y Y 1 B

q_0 B X X [Y] Y 1 B

q_3 B X X Y [Y] 1 B

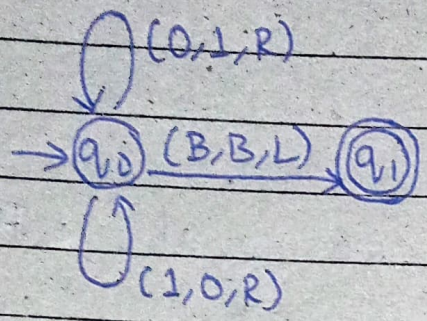
q_3 B X X Y Y [1] B

now, $\delta(q_3, 1)$ doesn't exist, so TM stops at $q_3 \notin F$

hence the string "00111" is not accepted.

Page No.
Date

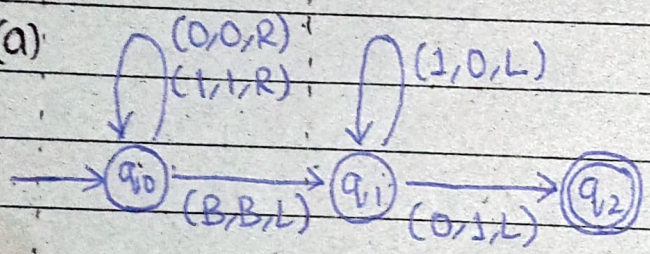
Q construct a TM for findings 1s compliment



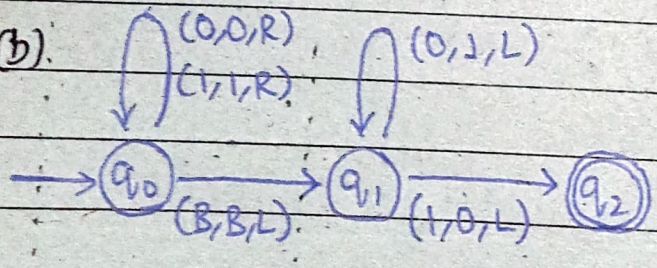
B	[1]	0	0	1	B	
B	0	[0]	0	1	B	
B	0	1	[0]	1	B	
B	0	1	1	[1]	B	
B	0	1	1	0	[B]	

Q construct TMs to increment/decrement a binary num by 1.

(a)



(b)



This part of both machine is responsible for taking the head pointer to the rightmost end of the string.

(a) incrementer

(b) decrementer

eg of increment:

(state = q_0)

B	[1]	0	0	1	B
B	1	[0]	0	1	B
B	1	0	[0]	1	B
B	1	0	0	[1]	B
B	1	0	0	1	[B]

(state = q_1)

B	1	0	0	[1]	B
B	1	0	[0]	0	B

(state = q_2)

B	1	[0]	1	0	B

(machine stops)

[1001 $\xrightarrow{(+1)}$ 1010]

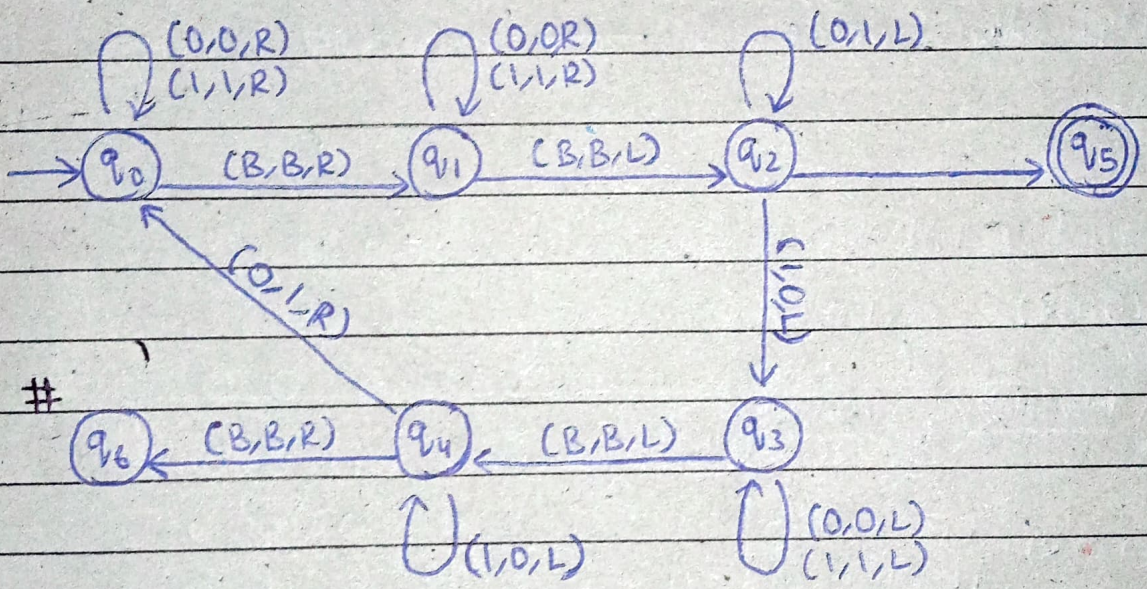
Page No:
Date

Q construct a TM to add two binary numbers

Logic :

```
while (b≠0)
    dcr(b)
    inr(a)
sum = a
```

thus, by combining the machines for increment & decrement we can make a TM that is capable of adding two binary numbers



Input string : B 0 1 1 0 B 0 1 0 1 B

Output string : B 1 0 1 1 B 1 1 1 1 B

explanation : $(0110)_2 + (0101)_2 = (1011)_2$

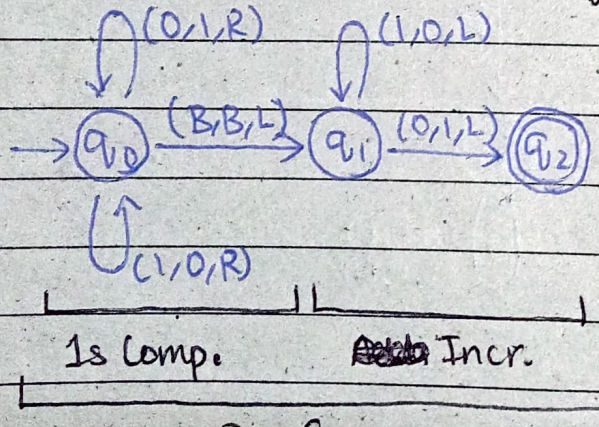
here q_6 is the overflow state

- increment + decrement ⟶ addition
- decrement + decrement ⟶ subtraction

- 1s compliment + increment ⟶ 2s compliment
- 2s compliment + addition ⟶ subtraction

- addition + decrement ⟶ multiplication
- multiplication + decrement ⟶ exponent

& construct a TM for finding 2s compliment



B [0] 1 0 0 B
B 1 [1] 0 0 B
B 1 0 [0] 0 B
B 1 0 1 [0] B
B 1 0 1 1 [B]
B 1 0 1 [1] B
B 1 0 [1] 0 B
B 1 [0] 0 0 B
B 1 1 0 0 B
(machine stops)

0100 ⟶ 1100

Page No.
Date

Context Sensitive Languages/Grammar:

A CSL is a language that can be defined by a CSG. Computationally, a CSL is equivalent to a linear bounded automaton. This means that every formal language that can be decided by an LBA is a CSL, and every CSL can be decided by an LBA.

example of CSL: $L = \{a^n b^n c^n : n \geq 1\}$

Linear Bounded Automata

An LBA is a non-deterministic TM with a tape of some bounded finite length, where the length depends on the length of initial input string.

An LBA can be represented by 8 tuples

Q : FNES of states
q_0 : initial state
F : FNES of final states
Σ : FNES of input symbols
Γ : FNES of tape symbols
M_L : Left end marker
M_R : Right end marker
δ : FNES of transitions

Page No.
Date

Church's Thesis

The church-turing thesis says that every solvable decision problem can be transformed into an equivalent Turing Machine problem. and,
Anything that can be done on any existing digital computer can also be done by a Turing machine.

several variations of Turing Machines:

→ two way infinite tape
→ multitape single head
→ multi tape multi head
→ single tape multihead
→ multi dimensional tape
→ non deterministic TM

It might seem that these variations are more powerful than the standard TM, however, it is not the case.

Relationship of computational power

DFA = NFA < DPDA < NPDA < LBA < TM = NTM

Page No.
Date

Universal TM

The Turing Machine is the machine level equivalent to a digital computer. However, the problem with TM is that a different machine must be constructed for every new computation to be performed

This the reason the Universal Turing Machine was introduced

The input of UTM includes

- The description of a machine M
- The input data

The UTM can then simulate M on the rest of the input tape. As a result, UTM can simulate any other machine

This leads to the idea of a "computer running another computer"

Page No.
Date

Primitive Recursive Functions

In computability theory, a primitive recursive function is a function that can be computed by a computer program and whose all loops are such that an upper bound of the no. of iterations of every loop can be determined before entering the loop.

A PR function takes a fixed number of arguments, each a whole number, and returns a whole number.

Some basic PR functions are:

- Zero Function: $Z(x) = 0$
- Successor Function: $S(x) = x + 1$
- Projection Function: $P_i^n(x_1, x_2, ..., x_n) = x_i$

More complex PR functions can be found/obtained by applying Composition & Recursion over these basic PR functions a finite number of times.

Page No
Date

Proving / Generating new Primitive Recursive Functions

For a given function $f()$, find functions $g()$ & $h()$ such that

- $f(x_1, x_2, \ldots, x_n, 0) = g(x_1, x_2, \ldots, x_n)$
- $f(x_1, x_2, \ldots, x_n, x_{n+1}+1) = h\left(x_1, x_2, \ldots, x_n, f(x_1, x_2, \ldots, x_n, x_{n+1})\right)$

If we are able to find $g()$ & $h()$, then $f()$ is recursive

If we $g()$ & $h()$ are primitive, then $f()$ is primitive recursive.

eg: $f_1(m,n) = m+n$

$\Rightarrow g(m) = f(m,0) = m+0 = m$

hence we can write

$g(m) = P_1^1(m) = m \quad \Rightarrow g() = P_1^1()$

$\Rightarrow h(m,n,f_1(m,n)) = f_1(m,n+1) = m+(n+1) = m+n+1$

hence we can write

$h(m,n,f_1(m,n)) = S(P_3^3(m,n,f_1(m,n)))$

$\Rightarrow h() = S(P_3^3())$

$\Rightarrow$ hence $f_1(m,n) = m+n$ is a primitive recursive function.

Page No.
Date

eg: $f_2(m,n) = m*n$

$\Rightarrow g(m) = f(m,0) = m*0 = 0$

$\Rightarrow g(m) = z(m) \quad \Rightarrow g() = z()$

$\Rightarrow h(m,n,f(m,n)) = f(m,n+1) = m*(n+1) = m*n + m$

$= f_2(m,n) + m = f_1(f_2(m,n), m)$

$= f_1(P_3^3(m,n,f(m,n)), P_1^3(m,n,f(m,n)))$

$\Rightarrow h(\ ) = f_1(P_3^3(), P_1^3())$

hence $f_2(m,n) = m*n$ is a primitive recursive function.

eg: $f_3(m,n) = m^n$

$\Rightarrow g(m) = f(m,0) = m^0 = 1$

$\Rightarrow g(m) = S(z(m)) \quad \Rightarrow g() = S(z())$

$\Rightarrow h(m,n,f(m,n)) = f(m,n+1) = m^{(n+1)} = m^n * m$

$= f_3(m,n) * m = f_2(f_3(m,n), m)$

$= f_2(P_3^3(m,n,f(m,n)), P_1^3(m,n,f(m,n)))$

$\Rightarrow h() = f_2(P_3^3(), P_1^3())$

hence $f_3(m,n) = m^n$ is a primitive recursive function.

Page No
Date

eg: $\lambda(a,b,c) = (ab)+c$

$\Rightarrow g(a,b) = \lambda(a,b,0) = (ab)+0 = a*b$
$\Rightarrow g(a,b) = f_2(a,b)$
$\Rightarrow g() = f_2()$

$\Rightarrow h(a,b,c,\lambda(a,b,c)) = \lambda(a,b,c+1) = (ab)+(c+1) = (ab+c)+1$
$= \lambda(a,b,c)+1 = S(\lambda(a,b,c))$
$= S(P_4^4(a,b,c,\lambda(a,b,c))$
$\Rightarrow h() = S(P_4^4())$

hence $\lambda(a,b,c) = (ab)+c$ is a PR function.

eg: $\lambda(a,b,c) = a+(bc)$

$\Rightarrow g(a,b) = \lambda(a,b,0) = a+(b*0) = a$
$\Rightarrow g(a,b) = P_1^2(a,b)$
$\Rightarrow g() = P_1^2()$

$\Rightarrow h(a,b,c,\lambda(a,b,c)) = \lambda(a,b,c+1) = a+(b*(c+1)) = a+bc+b$
$= \lambda(a,b,c)+b = f_1(\lambda(a,b,c), b)$
$= f_1(P_4^4(a,b,c,\lambda(a,b,c)), P_2^4(a,b,c,\lambda(a,b,c)))$
$\Rightarrow h() = f_1(P_4^4(), P_2^4())$

hence $\lambda(a,b,c) = a+(bc)$ is a PR function.